



Apache Kafka — Day 3

Administration, Security, Troubleshooting & Kafka Connect

8 HOURS

INTERMEDIATE → ADVANCED

OPERATIONS → SECURITY → DATA INTEGRATION

SECTION 1

Day 3: Operating & Securing Kafka

1

Day 1

Fundamentals · Architecture ·
Cluster

2

Day 2

Producer/Consumer · Topics ·
Partitions · Configuration

3

Day 3

Administration · Troubleshooting ·
Security · Kafka Connect ·
Enterprise Pipelines

Day 3 Learning Objectives

Operations

Avro Producer/Consumer, admin tools, common problems, tuning

Security

Authentication, authorization, ACLs, SSL encryption

Integration

Kafka Connect, Source & Sink connectors, File, JDBC, Cassandra, Elasticsearch



Why Message Format Matters

Producers and consumers may evolve independently. A field renamed or added can break downstream consumers.

- ❏ Enterprise challenge: we need to think about message structure, serialization, schema evolution, and producer/consumer compatibility.

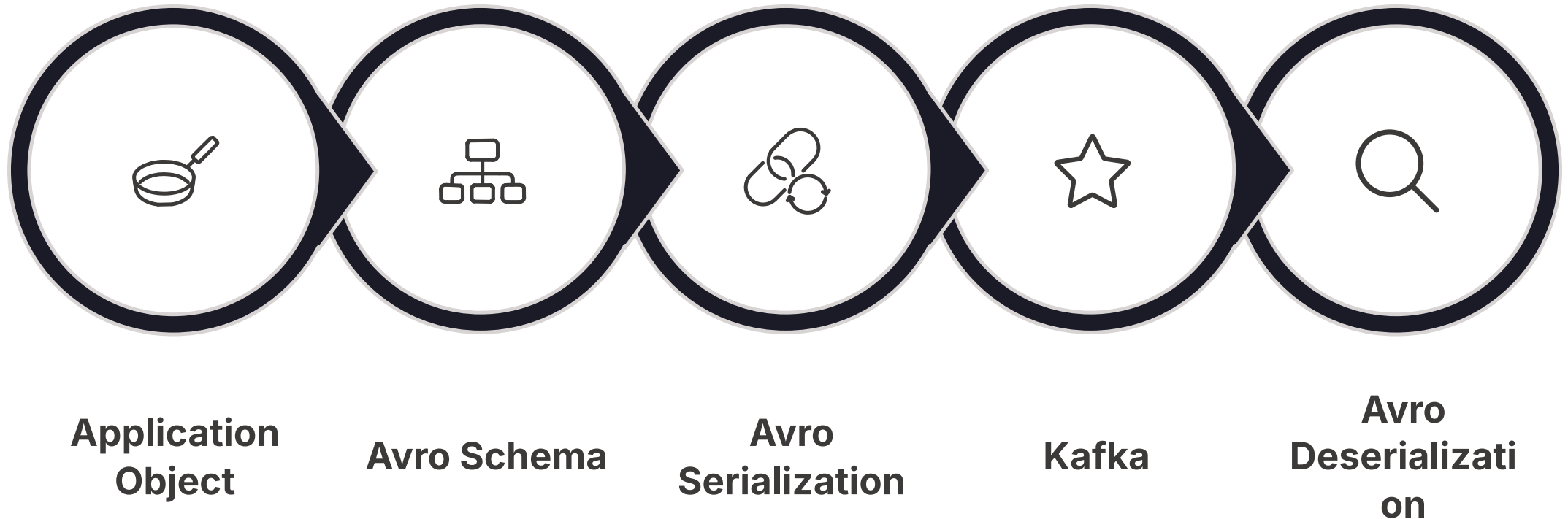
Before

```
customerId, name,  
accountType
```

After

```
customerId, fullName,  
accountType, status
```

Avro Producer & Consumer Flow



Avro provides a foundation for Schema Registry content in Day 4. Day 3 focuses on producer/consumer usage with a **PaymentEvent** schema: transactionId, accountId, amount, currency, timestamp.

What Does Kafka Administration Involve?

Cluster

Inspect brokers, review configurations, diagnose failures

Topics

Manage partitions, storage, and retention

Consumers

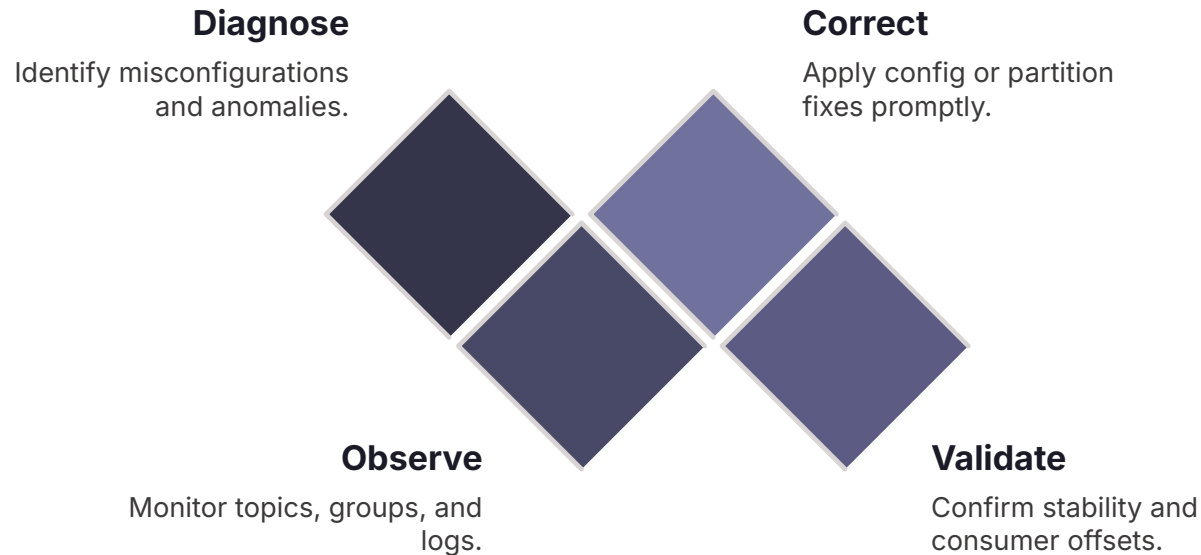
Monitor consumer groups, lag, and rebalancing

Security

Manage access controls and encryption



Kafka Administration Tools



Participants have already seen several CLI tools. Key administration areas include:

The key skill: understanding **what to inspect when something goes wrong.**

| **Topic Administration**

| **Consumer Group Administration**

| **Configuration Inspection**

| **Log Directory & Partition Operations**

Frequent Problems & Solutions

Producer is slow

Investigate: Producer / broker / network

Consumer is behind

Investigate: Consumer lag / processing

Broker unavailable

Investigate: Broker / host / configuration

Disk usage growing

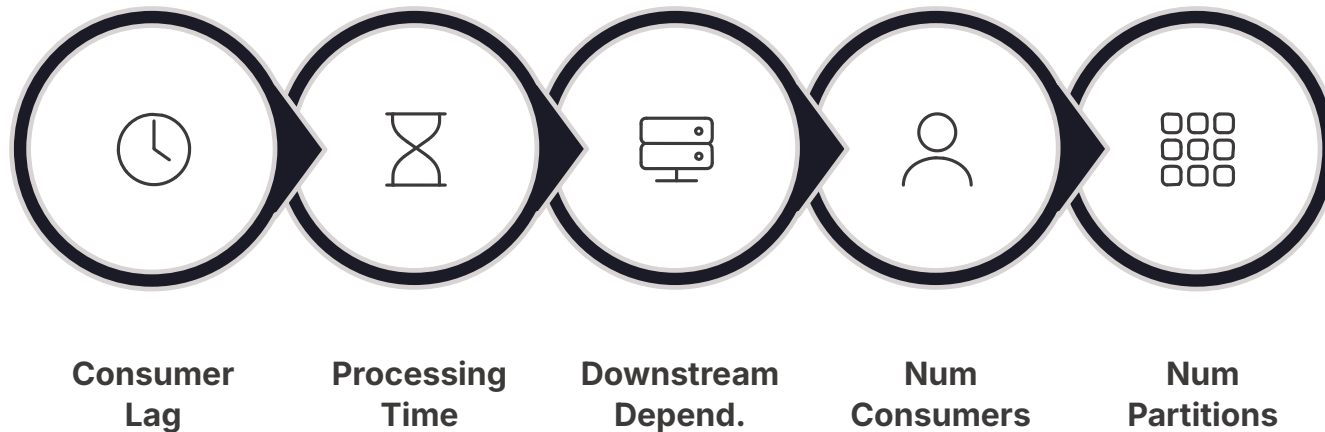
Investigate: Retention / log settings

Uneven workload

Investigate: Partition distribution

📄 Troubleshooting principle: SYMPTOM → METRIC/LOG → COMPONENT → ROOT CAUSE → ACTION

Troubleshooting Scenario: Consumer Lag



Scenario: Producer at 10,000 events/sec. Consumer at 6,000 events/sec. Lag continuously increases.

Potential causes:

- Slow application processing or database/API
- Insufficient consumer capacity
- Partition constraints or traffic spikes

❏ Kafka may be healthy while the **consumer application is the bottleneck.**

Troubleshooting Scenario: Disk Growth



Broker disk usage climbing: 60% → 70%
→ 82% → 91%

Investigate:

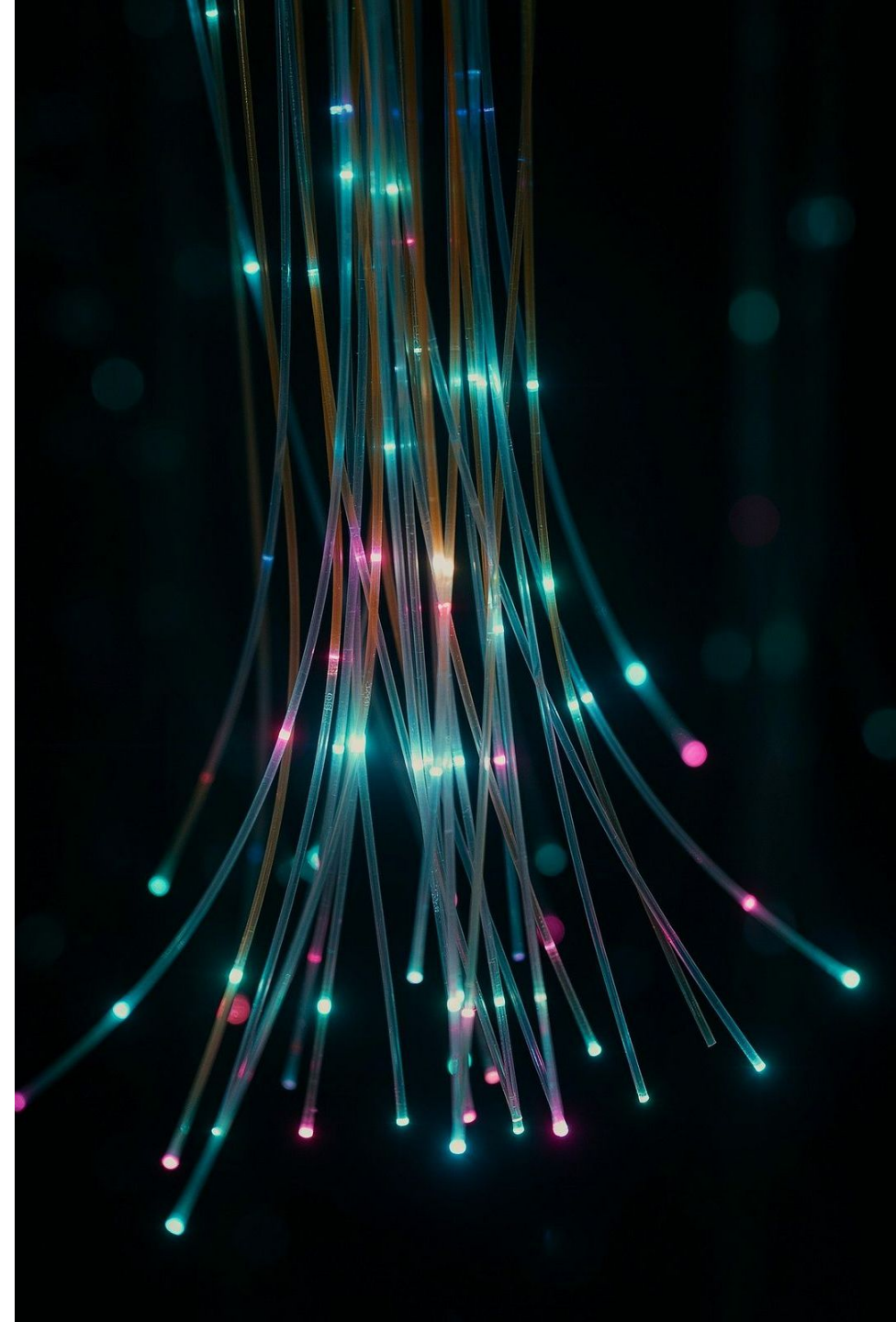
- Retention configuration:
`log.retention.ms`, `log.retention.bytes`
- Topic traffic and message size
- Partition count and log directories

❓ Is increasing disk capacity the solution, or are we retaining more data than the business requires?

Kafka Performance: Think End-to-End



Performance tuning must consider the **entire pipeline** — not just one setting or component.



Producer & Consumer Tuning

Producer

Key configurations from Day 2:

- `batch.size` + `linger.ms` → throughput/latency trade-off
- `compression.type` → lower network usage vs. CPU cost
- `acks`, `retries`, `buffer.memory`

Consumer

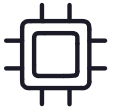
Potential bottlenecks:

- Slow business logic, database, or external API
- Insufficient consumers or too few partitions
- Error/retry loops

Scaling: More partitions → more consumer parallelism → more capacity.

❏ There is no universal "best Kafka configuration." Tune after measuring, not before.

Broker & OS Tuning



CPU

Is CPU becoming a bottleneck under compression or heavy load?



Memory

Is JVM heap and OS page cache sized appropriately?



Disk

Is disk throughput sufficient? Is storage balanced across brokers?



Network

Is the network saturated? Are partitions distributed effectively?

Kafka Across Data Centers



Data Center A → Kafka Cluster A → **replication** → Kafka Cluster B → Data Center B

Why? Disaster recovery, geographic distribution, data aggregation, migration.

Challenges:

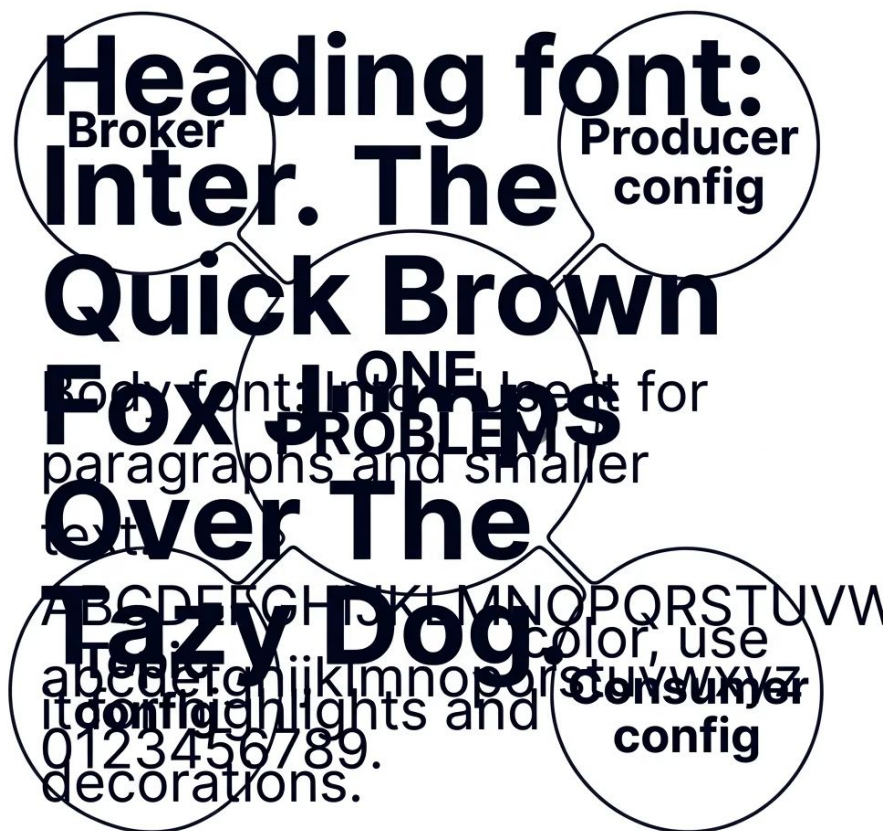
- Network latency and bandwidth
- Replication delay and failure handling
- Security across boundaries

Connects back to the **MirrorMaker/DR** introduction from Day 2.

Kafka Operational Best Practices

1	Topic Design Choose partitions based on scalability and workload requirements.
2	Producer Use appropriate keys, batching, compression, and delivery configuration.
3	Consumer Monitor lag, processing time, rebalancing, and failures.
4	Broker Monitor CPU, memory, disk, network, and partition distribution.
5	Operations Document and control configuration changes, retention, security, and capacity planning.

Kafka Configuration: Where Problems Hide

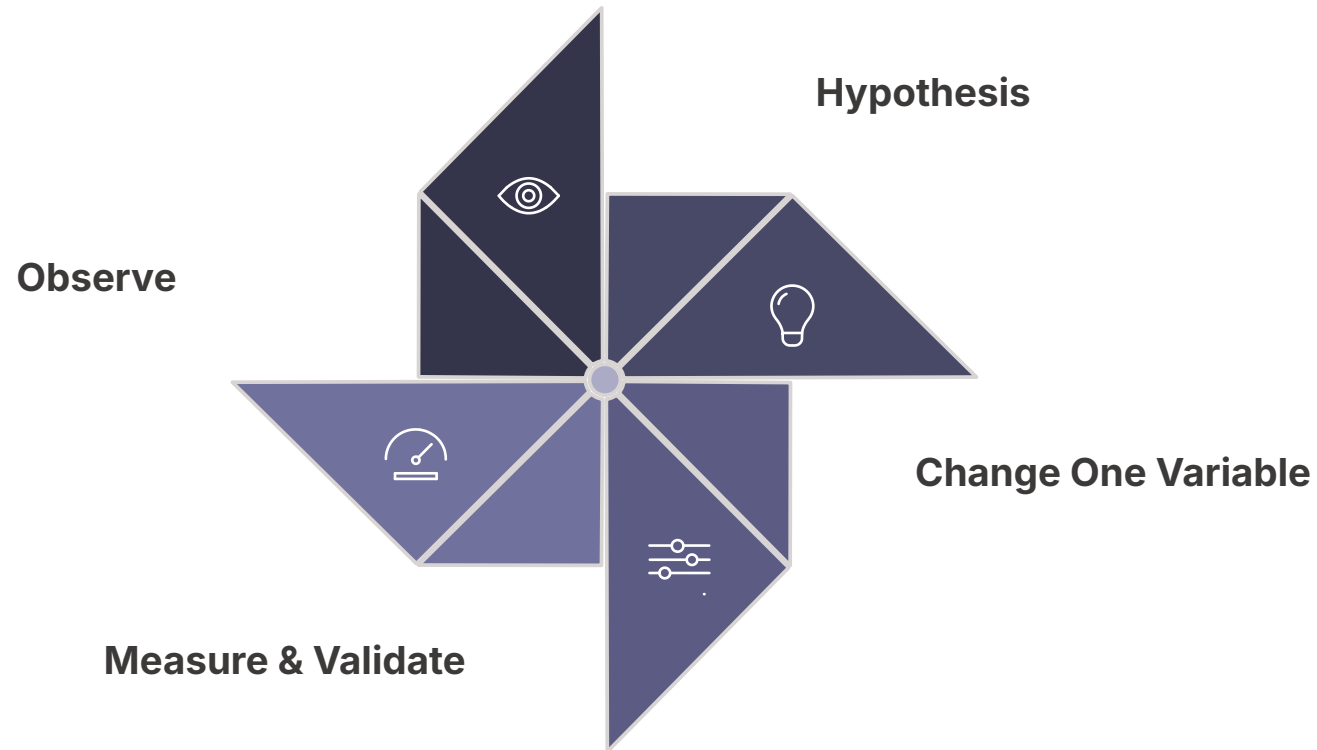


Configuration exists at multiple levels. A problem may come from any layer:

- | Broker Config
- | Producer Config
- | Consumer Config
- | Topic Config
- | Infrastructure

❏ Trainer rule: Always ask — "Which component owns this configuration?"

Structured Troubleshooting Approach



Define the Symptom Precisely

- ❌ "Consumer is slow"
- ✅ "Consumer lag increased from 5,000 to 500,000 in 20 minutes."

Check Evidence

Logs · Metrics · Configuration · Consumer lag · Broker state

Why Kafka Security Matters

Kafka may carry customer data, financial transactions, payments, and sensitive business events. Without controls, an unauthorized client can read sensitive events, publish malicious events, or access restricted topics.

Who are you?

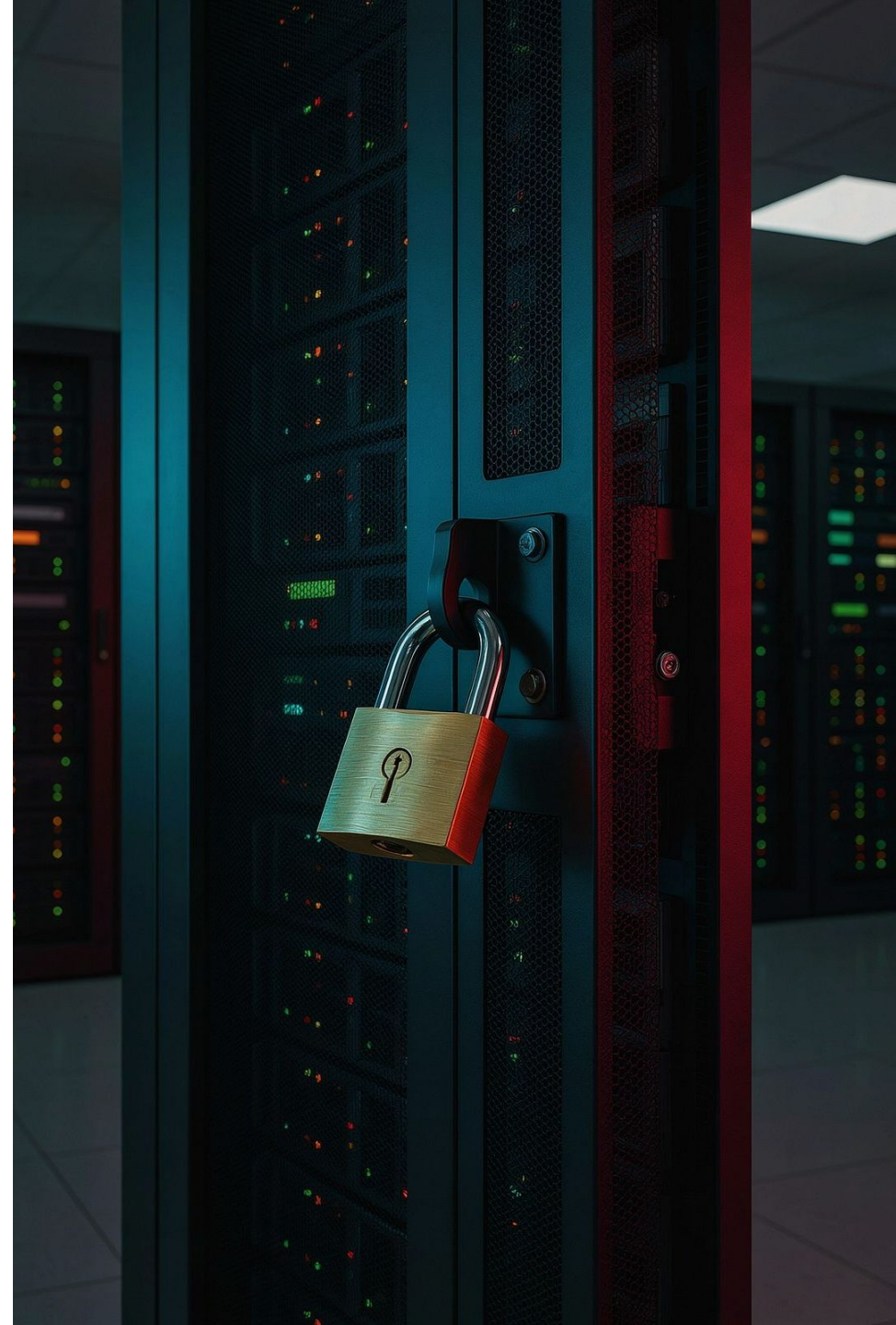
Authentication

What can you do?

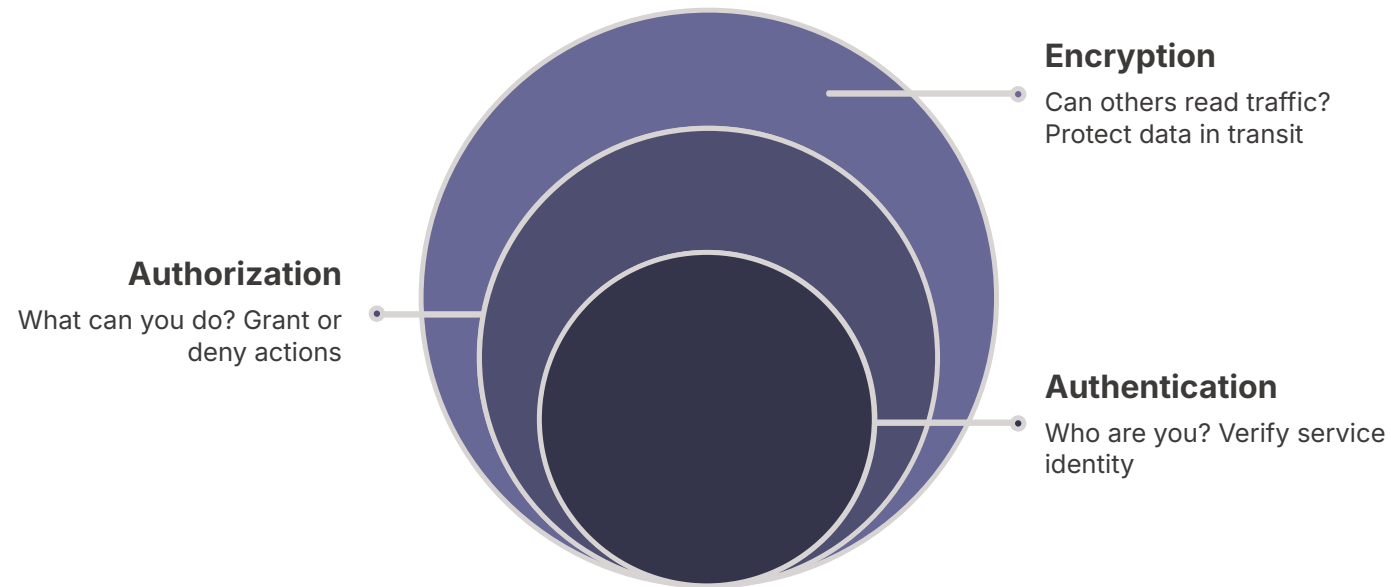
Authorization

Is traffic protected?

Encryption



Kafka Security Model



Example: Payment Service

Identity

payment-service

Allowed

WRITE → payment-events ✓

Not Allowed

READ → employee-data ×

Understanding Authorization

Authentication establishes identity. Authorization determines permissions.

Apply **Least Privilege** — give applications only the access required for their role.

Example: Producer A

WRITE orders ✓

READ orders ✕

DELETE orders ✕

Kafka ACLs

Access Control List = Principal + Operation + Resource

payment-service

Resource: payment-events

Operation: **WRITE**

fraud-service

Resource: payment-events

Operation: **READ**

ACLs allow administrators to precisely control which principals can perform which operations on which Kafka resources.

ACL Exercise

Scenario: Topic bank-transactions

Transaction Service

WRITE → bank-transactions

Fraud Service

READ → bank-transactions

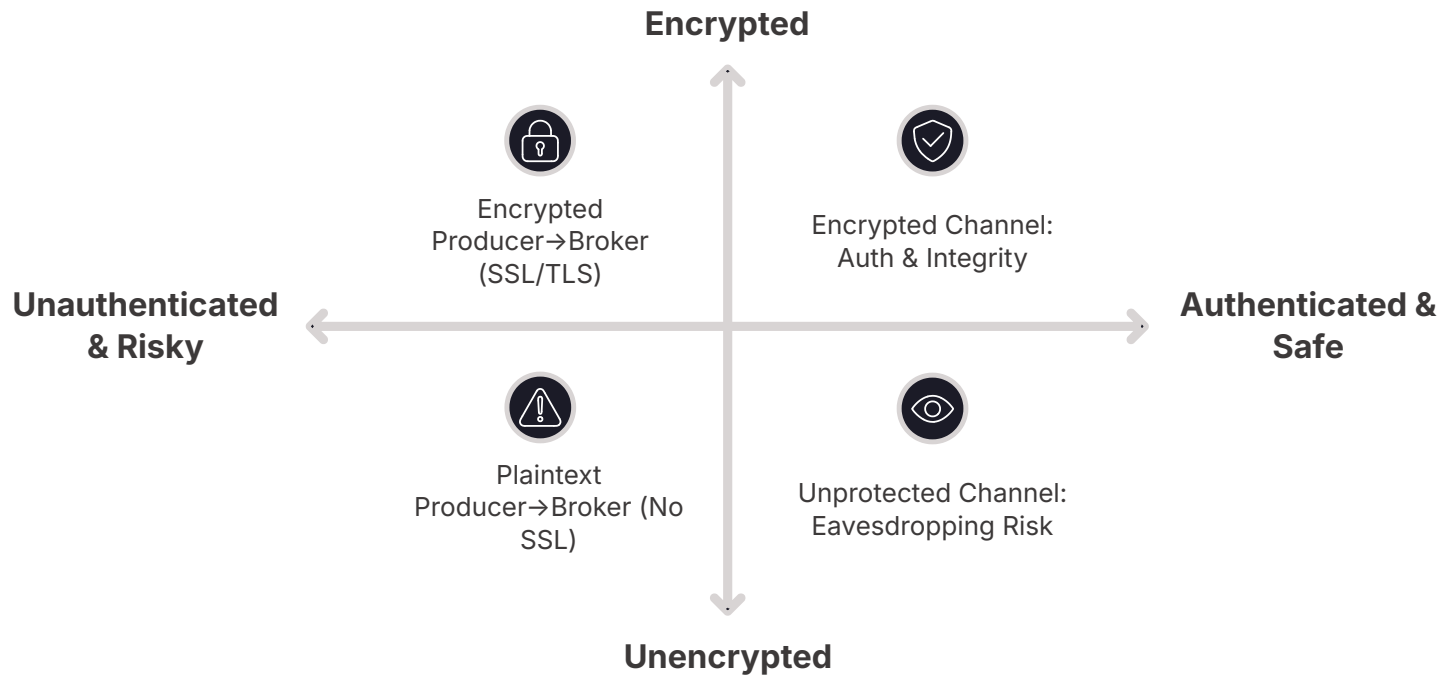
Analytics Service

READ → bank-transactions



Configure access, then test: allowed operation ✓ · unauthorized operation × · observe the result.

Wire Encryption Using SSL

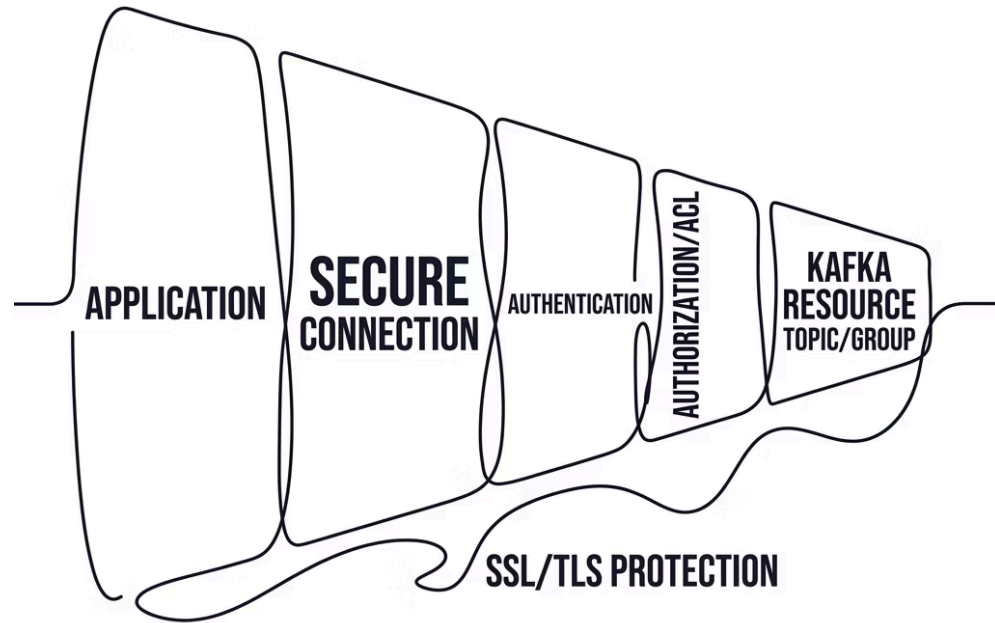


SSL/TLS protects data in motion between all Kafka components:

- Producer ↔ Broker
- Consumer ↔ Broker
- Broker ↔ Broker

The TOC explicitly includes an **SSL exercise** to reinforce this concept hands-on.

Security Architecture



Security Mental Model

1 Identity

Authentication establishes who is connecting.

2 Permission

ACLs define what each principal may do.

3 Encryption

SSL/TLS protects traffic in transit.

Hands-on Lab 1: Kafka Security

Part A — ACL

Create topic `secure-orders`. Define producer/consumer access. Test authorized ✓ and unauthorized ✕ operations.



Part B — SSL

Configure secure communication. Verify producer/consumer connectivity over encrypted channel.

✓ Learning outcome: Participants see the difference between **Access Control** vs **Transport Encryption**.

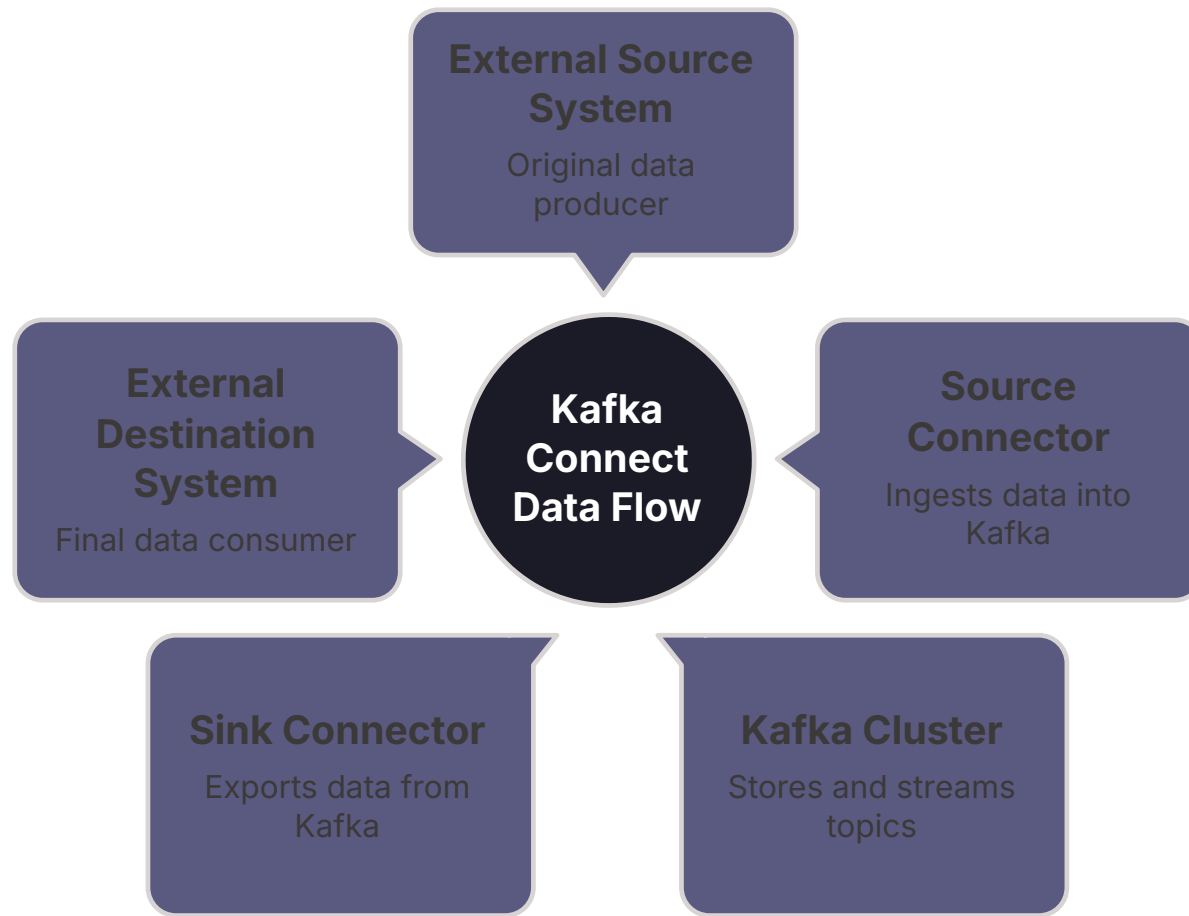
The Data Integration Problem

Moving data between MySQL, PostgreSQL, Elasticsearch, Cassandra, and Kafka requires custom code for every integration — creating significant maintenance burden.

- ❏ **Kafka Connect** addresses this problem with a reusable connector framework.



What Is Kafka Connect?



Kafka Connect is a framework for moving data between Kafka and external systems — without writing every integration from scratch.

The Day 3 module covers:

- Kafka Connect overview and architecture
- Implementing a pipeline end-to-end

Connector implementations handle common integration patterns so teams focus on configuration, not code.

Source vs Sink Connector

Source Connector

Moves data **into Kafka**.

Examples: File, JDBC, Cassandra-related integration

Database → Source Connector → Kafka

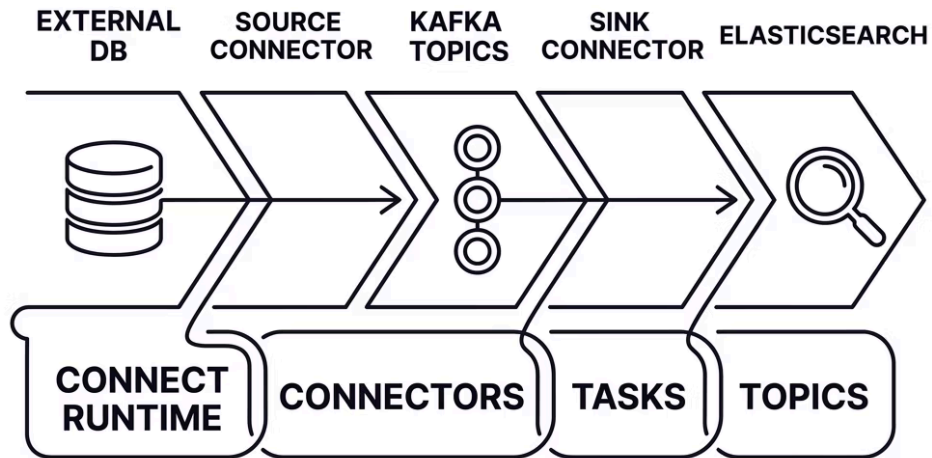
Sink Connector

Moves data **out of Kafka**.

Example: Kafka → Elasticsearch

Kafka → Sink Connector → External System

Kafka Connect Architecture



Key Concepts

Connector

Integration definition and logic

Connect Runtime

Manages connectors and tasks

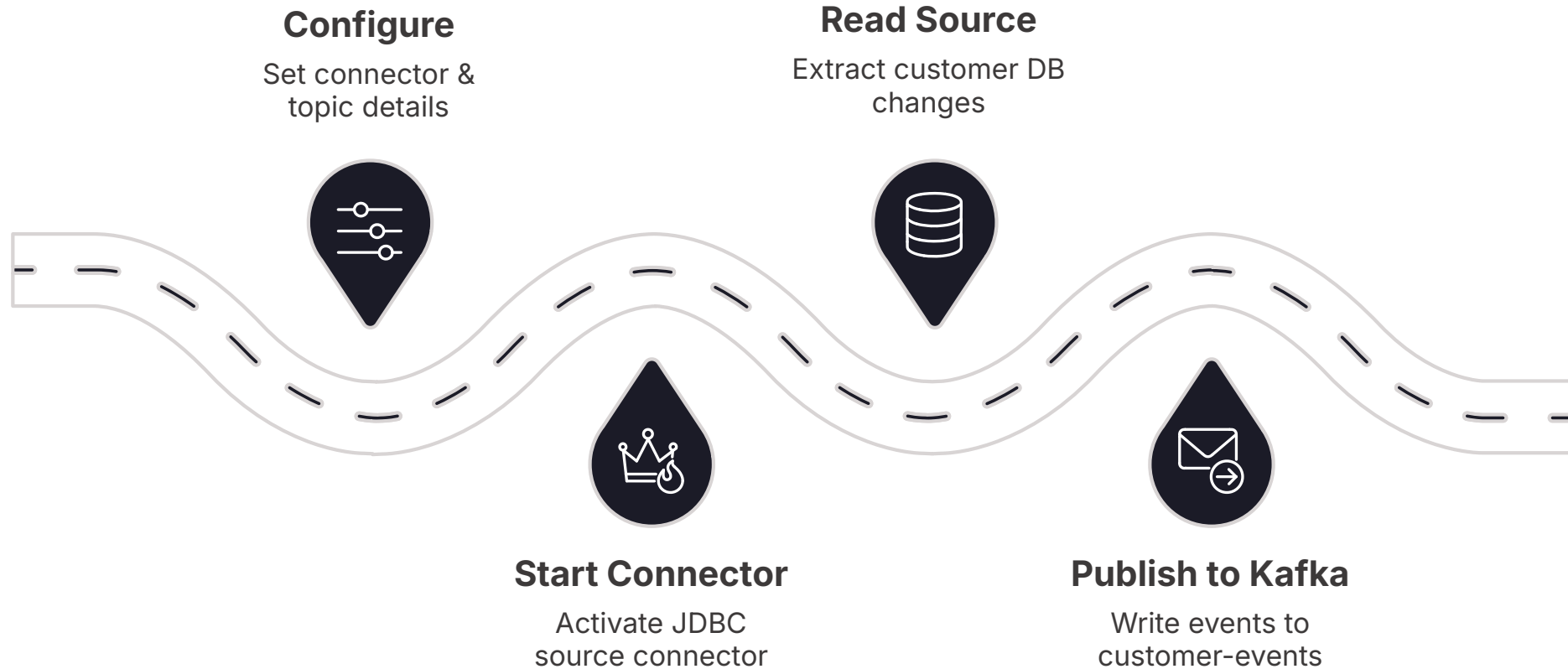
Task

Unit of work executing connector activity

Topics

Data flows through standard Kafka topics

Implementing a Data Pipeline



Example: Customer Database → JDBC Connector → customer-events topic → Kafka → Analytics + Elasticsearch Connector → Elasticsearch

Kafka Connect Use Cases & Connectors



File Connector

File → Kafka. Useful for understanding basic connector behavior.



JDBC Connector

Relational Database → Kafka. Common enterprise integration pattern.



Cassandra

Kafka ↔ Cassandra. Bidirectional depending on connector direction.



Elasticsearch

Kafka → Elasticsearch. Typical for search and analytics workloads.

Hands-on Lab 2: Build a Kafka Connect Pipeline

1

Exercise A — File

Input File → File Source
Connector → Kafka Topic. Verify
records using a consumer.

2

Exercise B — JDBC

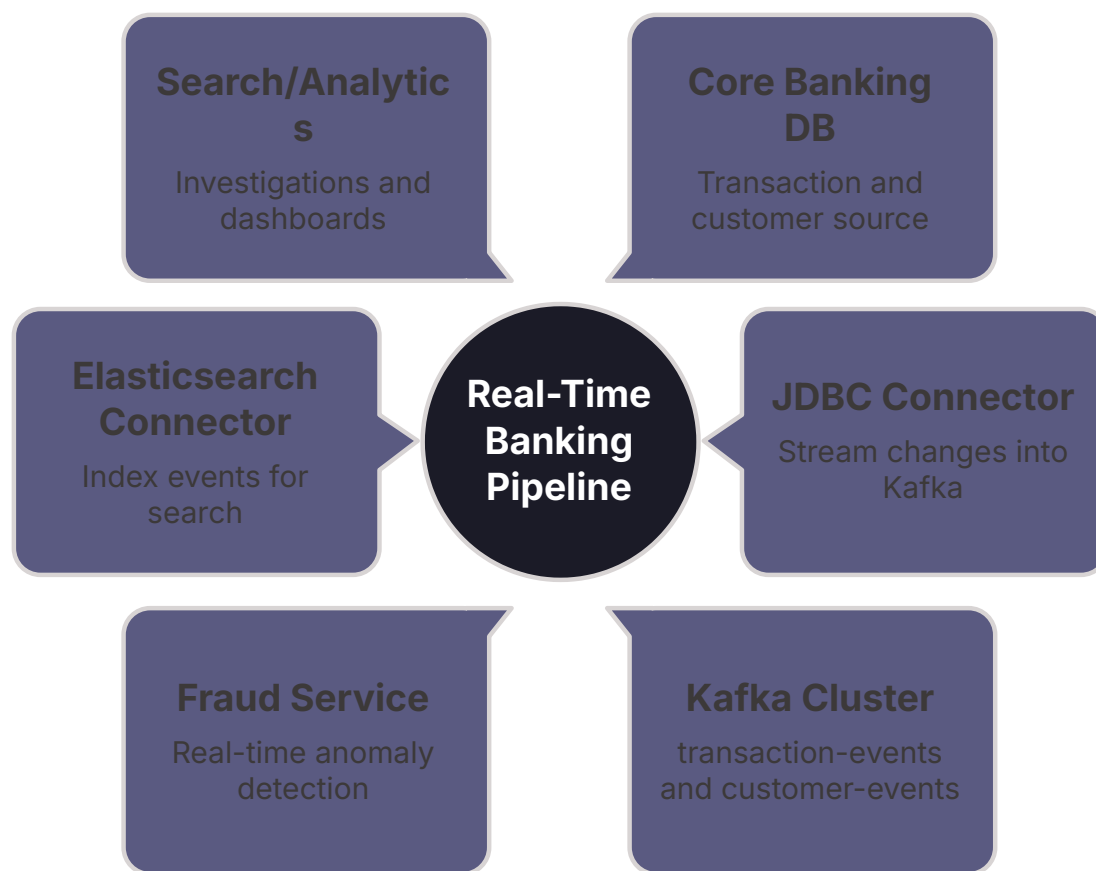
Database → JDBC Connector →
Kafka Topic. Observe records
arriving in Kafka.

3

Exercise C — Sink

Kafka → Connector → External
Target. Includes the explicit
Elasticsearch connector
exercise.

Case Study: Real-Time Banking Data Platform



Security Layer

- Authentication per application
- Authorization / ACL per topic
- SSL for all communication

Operations Monitoring

- Broker health · Consumer lag
- Connector behavior · Disk utilization
- Configuration · Security failures

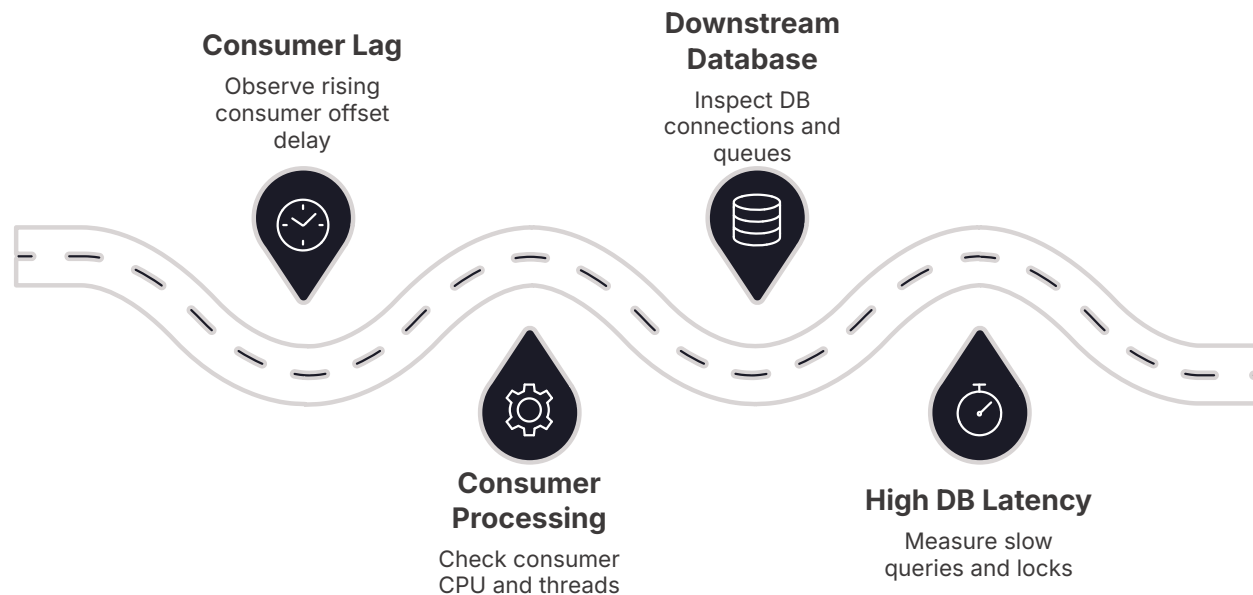
Day 3 Troubleshooting Challenge

Incident

Producer → Kafka → Consumer → Database

- Producer throughput: normal ✓
- Broker: operational ✓
- Consumer lag: **increasing rapidly** ×
- Database response time: **elevated** ×

❓ Where is the likely investigation path? Kafka? Consumer? Network? Database? Partitions?



📌 Key lesson: Kafka troubleshooting requires **end-to-end system thinking**.

Day 3 Complete Mental Model

Kafka Operations

- Avro Producer/Consumer
- Administration
- Troubleshooting & Tuning
- Best Practices & Configuration

Security

- Authentication
- Authorization & ACLs
- SSL Encryption

Data Integration

- Kafka Connect
- Source & Sink Connectors
- File, JDBC, Cassandra
- Elasticsearch

✓ Day 3 complete — you can now operate, secure, troubleshoot, and integrate Kafka at an enterprise level.